Generation of nonrecursive n-bit pseudorandom numbers by two times (n-bit)×(n-bit) multiplication ($n = 64, 128, 192, ..., 16384$)

Hirotake YAGUCHI

Abstract

We show that we can generate nonrecursive n-bit pseudorandom numbers by two times (n-bit)×(n-bit) multiplication and succeeding extraction of an n-bit integer from the result of multiplication. The algorithm, which we call MmS, is described using the functions defined by $T_{2^k}(X,Y) = 2^k XY - \lfloor 2^k XY \rfloor + 1$, X, $Y \in$ [1,2), and $\check{T}_{2^k}(X) \equiv T_{2^k}(X,X)$. We observe and consider mathematically the condition that $T_{2^k}(X,Y)$ and succeeding repetition of $\check{T}_{2^k}(X)$ generates random numbers, and see why our algorithm MmS is independent of the value of n. We also show the way of reducing the times of multiplication from two times to one time.

1 Introduction

We showed in [3] a simple algorithm of generating nonrecursive n-bit pseudorandom numbers, which is described by β-transformation $T_\beta(X) = \beta X - \lfloor \beta X \rfloor + 1$, $X \in [1,2)$, $\beta > 1$. Here we show another simple algorithm which consists of two times (n-bit)×(n-bit) multiplication and taking out an n-bit integer from the result of multiplication. The algorithm can be described using functions defined by $T_{2^k}(X,Y) = 2^k XY - \lfloor 2^k XY \rfloor + 1$, X, $Y \in$ [1,2), and $\check{T}_{2^k}(X) \equiv T_{2^k}(X,X)$.

In the next section we state the algorithm of generating nonrecursive random numbers(RNs), which we call **MmS**(**M**ultiply and **m**iddle-**S**quare), and then test randomness of RNs generated by MmS. The test was proceeded using TestU01 by L'Ecuyer & Simard [1]. The way of reducing times of (n-bit)×(n-bit) multiplication from two times to one time is stated in this section. In §3 we show that the algorithm of MmS is described by the fixed-bit-length computation of $T_{2^k}(X,Y)$ and $\check{T}_{2^k}(X)$ ('fixed-bit-length' means that a result of computation is truncated to an n-bit number). In §4, it is shown that *un*fixed-bit-length computation of $T_{2^k}(X,Y)$ and $\check{T}_{2^k}(X)$ also generates RNs ('unfixed-bit-length' means that a result of computation is not truncated to n-bit). This result allows us to put the influence of truncation aside when analyzing the algorithm of MmS mathematically . In §5, by modifying the algorithm of MmS a little, we observe the relation between the times of repetition of T_{2^k} and the randomness. In the last section we first consider mathematical conditions that repetition of $\check{T}_{2^k}(X)$ generates RNs, and then see why our MmS, which is described by $T_{2^k}(X,Y)$ and $\check{T}_{2^k}(X)$, $k = \frac{n}{2} - 1$, works well regardless of the value of n. In this paper we frequently quote and modify expressions in [3] without any mention for the sake of self-containedness of this paper.

2 Generation and test of nonrecursive n-bit random numbers

2.1 Generation of random numbers (MmS)

We generate nonrecursive random numbers(RNs) using the following simple algorithm (S0)-(S5):

(**S0**) Prepare two n-bit integers X and Y whose MSB(Most Significant Bit) are 1.

(**S1**) Multiply X and Y, and obtain a $2n$-bit number XY.

(**S2**) Take out n bits from just the middle of XY and write m.

(**S3**) Let M be the number obtained from m by setting the MSB to be 1. Then square M, and obtain a $2n$-bit integer MM.

(**S4**) Take out an n-bit integer z from just the middle of MM, and use it($= z$) as a random number.

(**S5**) To generate a sequence of random numbers z_0, z_1, z_2, ..., nonrecursively, vary X and Y in (S0) such as (X_0, Y_0), (X_1, Y_1), (X_2, Y_2), ..., and apply (S0)-(S4) to each (X_i, Y_i).

[We call the algorithm stated above the **MmS** (**M**ultiply and **m**iddle-**S**quare) for convenience sake.]

Remark 1. Considering z in (S4) as m in (S2), we can repeat (S3)-(S4) arbitrary times and use the last z as a RN. This kind of repetition is used in §5 and §6.

The n-bit integers (X_i, Y_i) in (S5) are defined as follows:

(i) Let e be Napier's constant $2.7182\cdots$. The hexadecimal representation of the number $(1+\frac{e}{10})\times 8(= 10.1746254\cdots)$ is 0xa.2cb4411 ba257552 232c1139 a172a5c9 $\cdots$ (0x means hexadecimal). Set the n-bit integer $\boldsymbol{B}$(base) to be the leading(first) n bits of the number 0xa.2cb44 11$\cdots$.

(ii) Let p, q, r and s be 64-bit prime numbers defined by

$$p = \texttt{0x7fffffffffffffe7}, \quad q = \texttt{0x7fffffffffffff5b},$$
$$r = \texttt{0x39f750241c2d5d33}, \quad s = \texttt{0x32f50fee9b2a32bb}.$$

Then define 64-bit integers $\boldsymbol{P_i}$ and $\boldsymbol{Q_i}$ as

$$P_i = r \times i \bmod p, \quad Q_i = s \times i \bmod q\ .$$

(These numbers were borrowed from [2]. Notice that MSBs of p, q, r, s and so P_i, Q_i are all 0.)

(iii) Finally, as X_i (resp. Y_i), we take XOR(exclusive OR, $\oplus$) of B and P_i (resp. Q_i) at just the middle of B. For example, if $n = 64$ and $i = 3$, then $B = \texttt{0xa2cb4411ba257552}$, $P_3 = \texttt{0x2de5f06c548817b2}$, and hence

$$X_3 = B \oplus P_3 = (\texttt{a2cb4411:ba257552}) \oplus (\texttt{2de5f06c:548817b2})$$
$$= (\texttt{8f2eb47d:eead62e0})\ .$$

Remark 2. When the current P_i is already known, the next P_{i+1} is computed easily as $P_{i+1} = P_i + r$ or $P_{i+1} = P_i + r - p$. This is the same for Q_i. The period of (P_i, Q_i) is pq.

Remark 3. The integers B, p, q, r, s are variable. For example in the case of $n = 8192$ and 16384 we used the leading n bits of $\pi - 2.5 = 0.64159\cdots =$ 0x0.a43f6a... to define B.

2.2 Test of randomness of generated numbers

Randomness of numbers $\{z_i\}_i$ generated in (S5) of MmS was tested by TestU01 by L'Ecuyer & Simard [1] in two ways: One is to test all bits of z_i (write **[Obj:A]** for short), and another is to test the leading 64 bits of z_i (write **[Obj:B]** for short).

[TestU01 by L'Ecuyer & Simard] We applied the battery Crush for $n = 64$, 128, 192, 256, 384, 512, 1024, 2048 and 4096. For $n = 8192$ and 16384 the battery SmallCrush was applied considering the time required. The Crush consists of 96 items of tests and generates 144 p-values. If the p-value of a test is out of the range [0.001, 0.9990], we further apply additional tests of three times, and if two or three p-values are out of the

range, we decide that $\{z_i\}_i$ can not pass that test. We treat p-values of SmallCrush in the same way. Then the result was that all tests were passed for $n = 64$, 128, 192, 256, 384, 512, 1024, 2048, 4096, 8192, 16384 and for [Obj:A], [Obj:B] (see also Table 1 in §4). (It took over 11 days totally to complete the Crush test applied to [Obj:B] of $n = 4096$.)

2.3 Reducing the times of multiplication from two times to one time

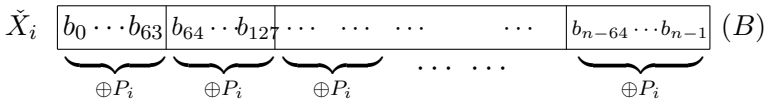

Figure 1: Taking XOR at every 64 bits of B

If $(X_i, Y_i)_i$ in (S5) is *well-'pre*randomized', we can use m_i in (S2) as a random number directly, and hence (S3) and (S4) can be omitted. In fact, let $\check{X}_i$ be the integer obtained from XOR of B and P_i in such a way that XOR is taken at every 64 bits of B (see Figure 1). Define $\check{Y}_i$ in the same way as $\check{X}_i$ using B and Q_i. Then the sequence $\{\check{m}_i\}_i$ obtained from $(\check{X}_i, \check{Y}_i)_i$ using (S0)-(S2) of MmS passes the Crush test stated in §2.2. Thus we can reduce the times of (n-bit)$\times$(n-bit) multiplication to generate RNs from two times to one time. This idea was thought out based on the result 'o' for $n = 64$, (S2) m_i in Table 1. We call this RNG(random number generator) the **p**rerandomized **MmS** (**p.MmS** for short).

3 Description of MmS by functions $T_{2^k}(X, Y)$ and $\check{T}_{2^k}(X)$

Let T_{2^k} , $k \geq 1$, be the function defined by $T_{2^k}(X, Y) = 2^k XY - \lfloor\, 2^k XY \rfloor + 1$, X, $Y \in$ [1,2), where $\lfloor q \rfloor$ is the largest integer not exceeding q. We frequently use the notation $\check{T}_{2^k}(X)$ instead of $T_{2^k}(X, X)$, that is, $\check{T}_{2^k}(X) = 2^k X^2 - \lfloor 2^k X^2 \rfloor + 1$, $X \in$ [1,2). (Here we have used the same symbols X and Y which were used in §2.1. The reason will be clear afterwards.) Below we give the graph of $T_{2^1}(X, Y)$, $\check{T}_{2^k}(X) \equiv T_{2^1}(X, X)$ and $\check{T}^2_{2^k}(X)$:

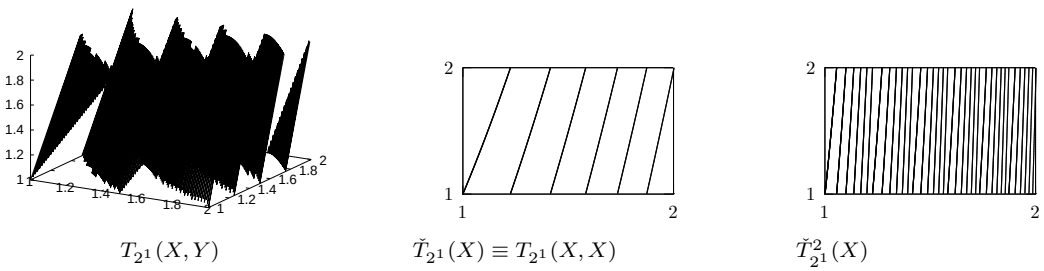

Figure 2: Graphs of T_{2^k} $(k = 1)$

Let the product $XY \in [1, 4)$ be represented as $XY = w_0 w_1 \,.\, w_2 w_3 w_4 \cdots$ in binary. ($w_0 w_1 = 01$ if $XY \in [1, 2)$, and $w_0 = 1$ if $XY \in [2, 4)$.) Then $2^k XY$ is represented as $2^k XY = w_0 w_1 \cdots w_{k+1} \,.\, w_{k+2} w_{k+3} \cdots$, that is, XY is shifted to the left by k w.r.t. the (fixed) zero point:

$$XY = w_0 w_1 \,.\, w_2 w_3 \cdots \quad \overset{shift\ left\ by\ k}{\Longrightarrow} \quad 2^k XY = w_0 w_1 \cdots w_{k+1} \,.\, w_{k+2} \cdots \,.$$

The final value $T_{2^k}(X, Y)$ is obtained by discarding(dropping) $w_0 w_1 \cdots w_k$ and replacing w_{k+1} with 1, that is, $T_{2^k}(X, Y) = 1.w_{k+2} w_{k+3} \cdots$.

Usually, in computer, integers are represented under a restricted bit-length, say n-bit, and results of multiplication of two n-bit integers are represented in $2n$-bit. Therefore

we follow this restriction when computing $T_{2^k}(X,Y)$ numerically. So suppose that n-bit X, Y and $2n$-bit XY are represented such as

$$X \equiv 1.u_1 \cdots u_{n-1}, \quad Y \equiv 1.v_1 \cdots v_{n-1} \quad \text{and} \quad XY \equiv w_0 w_1 \ . \ w_2 w_3 \cdots w_{2n-1}.$$

Then we have

$$2^k XY = w_0 \cdot \cdot w_{k+1} \ . \ w_{k+2} \cdots w_{2n-1} \quad \text{and} \quad T_{2^k}(X,Y) = 1.w_{k+2} \cdots w_{2n-1}.$$

Since $T_{2^k}(X,Y)$ is a $((2n-1)-k)$-bit number, in order to use this value as an n-bit variable X in $\check{T}_{2^k}(X) \equiv T_{2^k}(X,X)$, we truncate $T_{2^k}(X,Y)$ from $((2n-1)-k)$-bit to n-bit by cutting off the last $(n-1)-k$ bits including LSB(Least Significant Bit) :

$$T_{2^k}(X,Y) = 1.w_{k+2} \cdots w_{2n-1} \overset{truncate}{\Longrightarrow} 1.w_{k+2} \cdots w_{k+n} \text{ (the next } n\text{-bit } X).$$

(Notice that in the truncation above there occurs so-called rounding error due to cut-off of bits.) We call the computation above to get the next n-bit number $X = 1.w_{k+2} \cdots w_{k+n}$ from the initial n-bit numbers $X = 1.v_1 \cdots v_{n-1}$ and $Y = 1.v_1 \cdots v_{n-1}$, the **f**ixed-**b**it-**l**ength (discrete) **com**putation (**f.b.l.com** for short) of $T_{2^k}(X,Y)$. Remember that in the f.b.l.com of $T_{2^k}(X,Y)$, the leading $k+1$ bits $w_0 w_1 \cdots w_k$ of XY are dropped by a shift to the left, and the last $(n-1)-k$ bits $w_{k+n+1} \cdots w_{2n-1}$ of XY are cut off.

Now let's consider the special case $\boldsymbol{k = (n/2) - 1}$. In this case the flow of f.b.l.com of $T_{2^{(n/2)-1}}(X,Y)$ is written as follows:

$$\begin{array}{l}
X = 1.v_1 \cdots v_{n-1} \\ Y = 1.v_1 \cdots v_{n-1}
\end{array} \overset{\text{(S1)}}{\Longrightarrow} XY \equiv w_0 w_1 \ . \ w_2 \cdots w_{(n/2)} w_{(n/2)+1} \cdots w_{2n-1}$$

$$\overset{shift\ left\ by\ (n/2)-1\ bit}{\Longrightarrow} 2^{(n/2)-1} XY = w_0 w_1 \cdots w_{(n/2)} \ . \ w_{(n/2)+1} \cdots w_{2n-1}$$

$$\overset{drop\ the\ leading\ n/2\ bits}{\Longrightarrow} w_{(n/2)} \ . \ w_{(n/2)+1} \cdots w_{2n-1}$$

$$\overset{MSB=1}{\Longrightarrow} T_{2^{(n/2)-1}}(X,Y) = 1 \ . \ w_{(n/2)+1} \cdots w_{(3n/2)-1} w_{(3n/2)-1} \cdots w_{2n-1}$$

$$\overset{cut\ off\ the\ last\ n/2\ bits}{\Longrightarrow} 1.w_{(n/2)+1} w_{(n/2)+2} \cdots w_{(3n/2)-1} \quad \text{(the next } n\text{-bit } X).$$

Since the n bits $w_{(n/2)} w_{(n/2)+1} w_{(n/2)+2} \cdots w_{(3n/2)-1}$ above are located in just the middle of $2n$ bits $w_0 w_1 w_2 \cdots w_{(n/2)} w_{(n/2)+1} \cdots w_{2n-1}$ of XY, it is easy to see that the flow of f.b.l.com of $T_{2^{(n/2)-1}}(X,Y)$ given above corresponds to (S0), (S1), (S2) and the first half(where M is obtained) of (S3) in MmS. The latter half of (S3)(where MM is computed) and (S4) corresponds to f.b.l.com of $\check{T}_{2^{(n/2)-1}}(M)$ except that the MSB is not set to be 1(the MSB is used as it is). Thus we can say that MmS is described by f.b.l.com of $T_{2^k}(X,Y)$ and $\check{T}_{2^k}(X)$ with $k = (n/2) - 1$.

4 Unfixed-bit-length computation of T_{2^k} and randomness

The operation cut-off(rounding) which is used in the f.b.l.com(fixed-bit-length computation) of T_{2^k} makes complicated the mathematical analysis of algorithm of MmS. So we here introduce **uf.b.l.com** (**un**fixed-**b**it-**l**ength (discrete) **com**putation) of T_{2^k}. This is the one which is obtained from f.b.l.com by omitting the operation cut-off. Therefore, in uf.b.l.com, values $T_{2^k}(X,Y)$ and $\check{T}_{2^k}(X) \equiv T_{2^k}(X,X)$ obtained from (X,Y) and X respectively are used as the next values directly without cut-off. Thus uf.b.l.com is the (discrete) T_{2^k} function itself. (Notice that if we repeat uf.b.l.com, then the bit-length of result of T_{2^k} becomes longer and longer.) The uf.b.l.com has its meaning if the behavior of randomness is at the same level as that of f.b.l.com. Let $\{\zeta_i\}_i$ be the sequence of n-bit numbers which is obtained from uf.b.l.com in the same way as $\{z_i\}_i$ of f.b.l.com. To $\{\zeta_i\}_i$ we applied the Crush test for $n = 64$, 128, 192, 256,

384, 512 and for [Obj:B]. The result was that all tests were passed.

In Table 1 we give details of the result of Crush test for $\{z_i\}_i$ and $\{\zeta_i\}_i$. The table shows the number of tests in Crush whose p-values were out of the range $[10^{-15}, 1-10^{-15}]$. More precisely, $\Downarrow e$ (resp. $\downarrow e$, $\Uparrow e$, $\uparrow e$)) is the number of tests in Crush whose p-values were in $(0, 10^{-300})$ (resp. $[10^{-300}, 10^{-15})$, $(1-10^{-300}, 1)$, $(1-10^{-15}, 1-10^{-300}])$, and the symbol 'o' means that all p-values were in $[10^{-15}, 1-10^{-15}]$. The symbol $\leftarrow$ implies the test itself is the same as the left.

MmS	$n=64$		$n=128$		$n=192$		$n=256$		$n=384$		$n=512$	
[Obj:B]	f.b.l.	uf.b.l.	f.b.l.	uf.b.l.	f.b.l.	uf.b.l.	f.b.l.	uf.b.l.	f.b.l.	uf.b.l.	f.b.l.	uf.b.l.
(S2) m_i	o	$\leftarrow$	o	$\leftarrow$	$\Downarrow 2$	$\leftarrow$	$\Downarrow 1$	$\leftarrow$	$\Downarrow 6$	$\leftarrow$	$\Downarrow 5$	$\leftarrow$
(S4) z_i, ζ_i	o	o	o	o	o	o	o	o	o	o	o	o

Table 1: Randomness of MmS for $n = 64, \cdots, 512$

From the results of Crush test applied to $\{z_i\}_i$ and $\{\zeta_i\}_i$, we can think that the randomness of $\{\zeta_i\}_i$ and $\{z_i\}_i$ are at the same level. This result makes us possible to use uf.b.l.com of T_{2^k} when analyzing the mathematical background of MmS.

As a by-product of the test, we see that the sequence $\{m_i\}_i$ of (S2) of MmS for $n = 64$ passes the Crush test. This result suggested to us a way to the algorithm p.MmS stated in §2.3.

[**The simplified MmS** (s.MmS)] By the way we introduce here a simplified algorithm of MmS, which we call the **s**implified-**MmS** and write **s.MmS**. The s.MmS is the one obtained from MmS by setting $Y = X$ in (S0). Therefore s.MmS is described using only $\check{T}_{2^k}(X) \equiv T_{2^k}(X, X)$, $k = (n/2) - 1$. The sequence $\{z_i\}_i$ of s.MmS by f.b.l.com passes the Crush test stated in §2.2. In Table 2 we give details of the result for s.MmS which is obtained in the same way as Table 1.

s.MmS	$n=64$	$n=128$	$n=192$	$n=256$	$n=384$	$n=512$
[Obj:B]	f.b.l.	f.b.l.	f.b.l.	f.b.l.	f.b.l.	f.b.l.
(S2) m_i	o	$\downarrow 2 \Downarrow 10$	$\uparrow 7 \downarrow 2 \Downarrow 19$	$\uparrow 8 \downarrow 10 \Downarrow 26$	$\uparrow 5 \downarrow 7 \Downarrow 34$	$\uparrow 5 \downarrow 7 \Downarrow 26$
(S4) z_i	o	o	o	o	o	o

Table 2: Randomness of s.MmS for $n = 64, \cdots, 512$

From Table 2 we know that randomness of $\{m_i\}_i$ by s.MmS is worse than that of MmS. Results of (S4) ζ_i for uf.b.l.com of s.MmS, which we did not give in the table, were the same as those of f.b.l.com, that is, all 'o'.

From now the algorithm s.MmS which uses only $\check{T}_{2^k}(X) \equiv T_{2^k}(X, X)$ will appear frequently because of its simplicity. (The function $\check{T}_{2^k}$ might be called the Square-and-Shift Transformation(SST) on [1,2).)

5 Relations between times of repetition of T_{2^k} and randomness

In this section we modify the algorithm of MmS and s.MmS a little in such a way that k of T_{2^k} is $\frac{n}{s} - 1$, $s = 4$, 8 and 16. This is the case that the bits dropped in the computation of f(uf).b.l.com of T_{2^k} are the leading n/s bits, $s = 4$, 8 and 16. (In MmS and s.MmS, the s is 2.) We test and observe randomness of T_{2^k} when it is repeated.

5.1 The case only $\check{T}_{2^k}(X) \equiv T_{2^k}(X, X)$ is repeated

This case is corresponding to s.MmS. In order to see the condition that repetition of $\check{T}_{2^k}$ generates random numbers we vary ℓ of $\check{T}^{\ell}_{2^k}$ such as $\ell = 2, 3, \cdots$, and test the

randomness of numbers $\{\check{z}_i\}_i$ obtained from f.b.l.com of $\check{T}^{\ell}_{2^k}(X_i)$, $i = 0, 1, \cdots$, with ℓ fixed (see Remark 1). (Notice that the total times of (n-bit)×(n-bit) multiplication to compute $\check{T}^{\ell}_{2^k}(X_i)$ is ℓ, which we write as $\boldsymbol{\tau}_{\mathbf{mul}} = \ell$ for short.) Considering time required by tests, we restricted items of tests in Crush to

#11(BirthdaySpacings), #31(Gap), #37(Permutation) and #55(SumCollector).

These tests are ones whose p-values were extremely bad for many n when testing randomness of $\{m_i\}_i$ in order to make Tables 1 and 2. We applied these 4 tests to [Obj:B].

$\check{T}_{2^k}(X)$	$k = (n/4) - 1$					$k = (n/8) - 1$					$k = (n/16) - 1$				
$n \Longrightarrow$	256	384	512	1024		256	384	512	1024		256	384	512	1024	
XORpos $\Longrightarrow$	C	C	C	C	T	C	C	C	C	T	C	C	C	C	T
$\tau_{\text{mul}} = 2$	3	4	4	4	-	-	-	-	-	-	-	-	-	-	-
$\tau_{\text{mul}} = 3$	0	0	0	0	-	4	4	4	-	-	-	-	-	-	-
$\tau_{\text{mul}} = 4$	-	-	-	-	4	0	0	3	4	-	-	-	-	-	-
$\tau_{\text{mul}} = 5$	-	-	-	-	0	-	-	0	0	-	4	-	-	-	-
$\tau_{\text{mul}} = 6$	-	-	-	-	-	-	-	-	-	-	3	4	-	-	-
$\tau_{\text{mul}} = 7$	-	-	-	-	-	-	-	-	-	4	0	3	4	4	-
$\tau_{\text{mul}} = 8$	-	-	-	-	-	-	-	-	-	0	-	0	0	3	-
$\tau_{\text{mul}} = 9$	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-
$\tau_{\text{mul}} = 15$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	4
$\tau_{\text{mul}} = 16$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Table 3: Randomness when $\check{T}_{2^k}(X)$ is repeated

Table 3 is obtained using repetition of $\check{T}_{2^{(n/s)-1}}(X)$ for $n = 256$, 384, 512, 1024, and $s = 4$, 8, 16. Values in the table shows the number(0-4) of tests whose p-value were less than 10^{-15}. The notation XORpos=C and T imply that, when X_i is made by taking XOR of P_i with B, the position where XOR is taken is at the **C**enter of B and at the **T**ail of B respectively.

From the table we can observe the following (O1) - (O4):

(**O1**) For each n, as τ_{mul} increases, the number of tests whose p-values are less than 10^{-15} decreases, and becomes 0 around $\tau_{\text{mul}} = s/2$ if XORpos=C ($\tau_{\text{mul}} = s$ if XORpos=T).

(**O2**) The values $\tau_{\text{mul}} = s/2$ and s in (O1) are independent of n.

(**O3**) Since the first n/s bits are dropped in f.b.l.com of $\check{T}_{2^{(n/s)-1}}(X)$, total $\frac{n}{s} \times \ell$ bits are dropped in $\check{T}^{\ell}_{2^{(n/s)-1}}(X)$. Therefore in the case of $\tau_{\text{mul}} = s/2$ in (O2) total $\frac{n}{s} \times \frac{s}{2} = n/2$ bits are dropped regardless of the value of s. If $\tau_{\text{mul}} = s$, total $\frac{n}{s} \times s = n$ bits are dropped regardless of the value of s.

(**O4**) Comparing the result of XORpos=C and XORpos=T for $n = 1024$, we know that the value $\check{\ell}$ of τ_{mul} at which randomness appears apparently is strongly dependent of the position where XOR of P_i with B is taken. Roughly speaking, if $\check{\ell}_C$ and $\check{\ell}_T$ are $\check{\ell}$ of XORpos=C and T respectively, then $\check{\ell}_T$ is about two times larger than $\check{\ell}_C$ (see also Table 4).

5.2 The case firstly $T_{2^k}(X, Y)$ and then repetition of $\check{T}_{2^k}(X)$ are used

This case is corresponding to MmS. For a given (X, Y) we first compute $F = T_{2^k}(X, Y)$, and then repeat $\check{T}_{2^k}(X)$ $\ell - 1$ times starting with $X = F$. As an n-bit random number we use $\check{z}$ obtained from $\check{T}^{\ell-1}_{2^k}(F)$ (see Remark 1). (Note that the total times of (n-bit)×(n-bit) multiplication to get $\check{z}$ is ℓ, that is, $\tau_{\text{mul}} = \ell$.)

Table 4 is created in the same way as Table 3 using firstly $T_{2^{(n/s)-1}}(X, Y)$ and then repetition of $\check{T}_{2^{(n/s)-1}}(X)$, $s = 8$, 16, for $n = 256$, 512, 1024. In the table XORpos=CT

$T_{2^k}(X,Y)$	$k=(n/8)-1$									$k=(n/16)-1$								
$n \Longrightarrow$	256			512			1024			256			512			1024		
XORpos $\Longrightarrow$	CC	CT	TT	CC	CT	TT	CC	CT	TT	CC	CT	TT	CC	CT	TT	CC	CT	TT
$\tau_{\rm mul}=3$	4	4	-	4	4	-	4	-	-	-	-	-	-	-	-	-	-	-
$\tau_{\rm mul}=4$	0	0	-	0	3	-	2	4	-	-	-	-	-	-	-	-	-	-
$\tau_{\rm mul}=5$	-	-	-	-	0	-	0	0	-	4	-	-	-	-	-	-	-	-
$\tau_{\rm mul}=6$	-	-	4	-	-	-	-	-	-	3	4	-	-	-	-	-	-	-
$\tau_{\rm mul}=7$	-	-	0	-	-	4	-	-	4	0	0	-	4	4	-	4	4	-
$\tau_{\rm mul}=8$	-	-	-	-	-	0	-	-	0	-	-	-	0	0	-	0	3	-
$\tau_{\rm mul}=9$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-
$\tau_{\rm mul}=11$	-	-	-	-	-	-	-	-	-	-	-	4	-	-	-	-	-	-
$\tau_{\rm mul}=12$	-	-	-	-	-	-	-	-	-	-	-	3	-	-	-	-	-	-
$\tau_{\rm mul}=13$	-	-	-	-	-	-	-	-	-	-	-	0	-	-	4	-	-	-
$\tau_{\rm mul}=14$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	3	-	-	-
$\tau_{\rm mul}=15$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-	-	4
$\tau_{\rm mul}=16$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

Table 4: Randomness when $T_{2^k}(X,Y)$ and repetition of $\check{T}_{2^k}(X)$ are used (s =8,16)

implies that when X_i and Y_i are made by taking XOR of P_i and Q_i with B, the positions where XOR is taken are at the Center of B and at the Tail of B respectively. The expressions CC and TT express Center/Center and Tail/Tail respectively.

From Table 4 we know that properties (O1)-(O4) in §5.1 are also observed replacing XORpos=C and T with XORpos=CC and TT respectively. It should be noted here that

(**O5**) the result of setting XORpos=CT is almost the same as that of XORpos= CC. The (O5) says that when 'C' and 'T' are used simultaneously in $T_{2^k}(X,Y)$, then the position 'C' plays a more important role.

6 Mathematical consideration

In this section we give mathematical explanation to the observations (O1)-(O5) in §5, and consider the reason why the algorithm of MmS is independent of the value of n.

6.1 Mathematical consideration of repetition of $\check{T}_{2^k}(X) \equiv T_{2^k}(X,X)$

Let's write $\check{T}^\ell(X) \equiv \check{T}^\ell_{2^k}(X)$, $\ell = 1,2,\cdots$, and put $\check{T}^0(X) = X$. Then define an integer function $J_\ell(X)$ by

$$\check{T}^\ell(X) = 2^k(\check{T}^{\ell-1}(X))^2 - J_\ell(X), \quad J_\ell(X) = \lfloor 2^k(\check{T}^{\ell-1}(X))^2 \rfloor - 1. \tag{Eq1}$$

For a positive $\triangle(\ll 1)$, define $\check{T}^\ell(X+\triangle)$ and $J_\ell(X+\triangle)$ analogously. (If $\triangle$ is negative, then consider $X' \equiv X+\triangle$ and $X'+\triangle'$, $\triangle' = -\triangle$.) Let $\tilde{J}_\ell(X,\triangle)$ be defined by $\tilde{J}_\ell(X,\triangle) \equiv J_\ell(X+\triangle) - J_\ell(X)$. Now starting with $W_0(X,\triangle) = \triangle$ and $U_0(X,\triangle) = 0$, define inductively for $\ell = 1,2,\cdots$,

$$W_\ell(X,\triangle) = 2^k W_{\ell-1}(X,\triangle)\{2\check{T}^{\ell-1}(X) + W_{\ell-1}(X,\triangle)\} \quad \text{and} \tag{Eq2}$$

$$U_\ell(X,\triangle) = 2^k U_{\ell-1}(X,\triangle)\{2\check{T}^{\ell-1}(X) + 2W_{\ell-1}(X,\triangle) + U_{\ell-1}(X,\triangle)\} - \tilde{J}_\ell(X,\triangle).$$

Then it holds that

$$\check{T}^\ell(X+\triangle) = \check{T}^\ell(X) + W_\ell(X,\triangle) + U_\ell(X,\triangle). \tag{Eq3}$$

This relation is verified by induction

$$\begin{aligned}\check{T}^\ell(X+\triangle) &= \check{T}_{2^k}(\check{T}^{\ell-1}(X+\triangle)) = 2^k(\check{T}^{\ell-1}(X+\triangle))^2 - J_\ell(X+\triangle) \\ &= 2^k\{\check{T}^{\ell-1}(X) + W_{\ell-1} + U_{\ell-1}\}^2 - J_\ell(X+\triangle),\end{aligned}$$

and (Eq1).

In order to investigate the local behavior of $\check{T}^{\ell}(X)$, we assume that all values from $\check{T}^{\ell}(X)$ to $\check{T}^{\ell}(X+\triangle)$ are on the same line(branch) in the graph of $\check{T}^{\ell}$. In this case, since $\tilde{J}_j(X,\triangle)=0,\ j=1,2,\cdots,\ell$, and then $U_j(X,\triangle)=0,\ j=1,2,\cdots,\ell$, (Eq3) becomes

$$\check{T}^{\ell}(X+\triangle)-\check{T}^{\ell}(X)=W_{\ell}(X,\triangle). \tag{Eq4}$$

Therefore in the following, making use of $W_{\ell}\equiv W_{\ell}(X,\triangle)$, we investigate the relation between ℓ and randomness. From (Eq2) we have

$$\begin{aligned} W_{\ell} &= 2^{\ell(k+1)}W_0\left\{\prod_{j=0}^{\ell-1}\check{T}^{j}(X)\right\}\times\left\{\prod_{j=0}^{\ell-1}\{1+W_j/2\check{T}^{j}(X)\}\right\} \\ &= 2^{\ell(k+1)}\triangle\left\{\prod_{j=0}^{\ell-1}\check{T}^{j}(X)\right\}\times\left\{\prod_{j=0}^{\ell-1}\{1+W_j/2\check{T}^{j}(X)\}\right\}. \end{aligned} \tag{Eq5}$$

We here assume $k=(n/s)-1$, $s=4,8,16$, as before and $\triangle=2^{-(n/t)}$, $t=1,2$, which emulates roughly the position where the result of taking XOR of B and P_i appears. ($t=2$ and $t=1$ correspond to XORpos=C and T respectively). Let $\theta_{\ell,X}\in(0,1)$ be the number satisfying $\prod_{j=0}^{\ell-1}\check{T}^{j}(X)=(2^{\theta_{\ell,X}})^{\ell}$. Then (Eq4)-(Eq5) is rewritten as

$$\check{T}^{\ell}(X+\triangle)-\check{T}^{\ell}(X)=2^{\ell((n/s)+\theta_{\ell,X})-(n/t)}\left\{\prod_{j=0}^{\ell-1}\{1+W_j/2\check{T}^{j}(X)\}\right\}. \tag{Eq6}$$

Looking at Table 3 in §5.1 and (Eq6), we consider that the randomness appears apparently after the exponent of $2^{\ell((n/s)+\theta_{\ell,X})-(n/t)}$ in (Eq6) changes its sign from $-$(minus) to $+$(plus) when ℓ increases. Let $\check{\boldsymbol{\ell}}$ be the smallest integer ℓ satisfying

$$\ell((n/s)+\theta_{\ell,X})-(n/t)\geq 0\ \ (\Leftrightarrow\ \ell\geq\frac{s}{t}\left(1-\frac{(s/n)\theta_{\ell,X}}{1+(s/n)\theta_{\ell,X}}\right)\ ). \tag{Eq7}$$

Since $\check{\ell}$ is $\frac{s}{t}$ if $\frac{s}{n}\ll 1$, it is considered that the randomness appears apparently around $\tau_{\mathrm{mul}}=\frac{s}{t}(=\check{\ell})$. The expression $\check{\ell}=\frac{s}{t}$ is independent of n, and immediately implies $\frac{n}{s}\times\check{\ell}=\frac{n}{t}$. (Figure 3 is an image of the relation $\frac{n}{s}\times\check{\ell}=\frac{n}{t}$.)

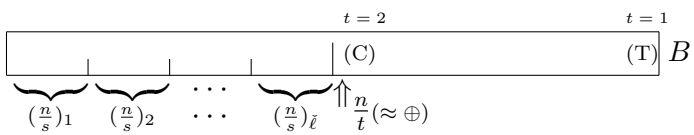

Figure 3: An image of $(n/s)\times\check{\ell}=(n/t)$

These expressions explain (O1) - (O4) in §5.1 easily. In fact, $\check{\ell}=\frac{s}{t},\ t=1,2,$ explains (O1), (O2) and (O4), and the expression $\frac{n}{s}\times\check{\ell}=\frac{n}{t},\ t=1,2,$ explains (O3).

6.2 The case $T_{2^k}(X,Y)$ and repetition of $\check{T}_{2^k}(X)\equiv T_{2^k}(X,X)$ are used

Here we follow the line given in §6.1. First let F and $F_{\triangle}$ be the numbers defined by

$$F=T_{2^k}(X,Y)\ \text{ and }\ F_{\triangle}=T_{2^k}(X+\triangle_X,Y+\triangle_Y).$$

The notation $\check{T}^{\ell}(X)\equiv T^{\ell}_{2^k}(X,X)$ is also used in this subsection. Then define an integer function $K_{\ell}(F),\ \ell=1,2,\cdots,$ by

$$\check{T}^{\ell}(F)=2^k(\check{T}^{\ell-1}(F))^2-K_{\ell}(F),\quad K_{\ell}(F)=\lfloor 2^k(\check{T}^{\ell-1}(F))^2\rfloor-1. \tag{Eq1'}$$

For $F_{\triangle}$ above, define $\check{T}^{\ell}(F_{\triangle})$ and $K_{\ell}(F_{\triangle})$ analogously. Let $\tilde{K}_{\ell}$ be the difference of $K_{\ell}(F_{\triangle})$ and $K_{\ell}(F)$, that is, $\tilde{K}_{\ell}=K_{\ell}(F_{\triangle})-K_{\ell}(F)$. Now starting with

$$\tilde{W}_1=2^k(X\triangle_Y+Y\triangle_X+\triangle_X\triangle_Y)\ \text{ and}$$

$$\tilde{U}_1=-\{\lfloor 2^k(X+\triangle_X)(Y+\triangle_Y)\rfloor-\lfloor 2^kXY\rfloor\}\ (\equiv-\tilde{K}_0),$$

define $\tilde{W}_{\ell}$ and $\tilde{U}_{\ell}$ inductively for $\ell=2,3,\cdots$ by

$$\tilde{W}_{\ell}=2^k\tilde{W}_{\ell-1}\{2\check{T}^{\ell-2}(F)+\tilde{W}_{\ell-1}\}\qquad\text{and} \tag{Eq2'}$$

$$\tilde{U}_\ell = 2^k \tilde{U}_{\ell-1}\{2\check{T}^{\ell-2}(F) + 2\tilde{W}_{\ell-1} + \tilde{U}_{\ell-1}\} - \tilde{K}_{\ell-1}.$$

Then it holds that for $\ell = 1, 2, \cdots$,

$$\check{T}^{\ell-1}(F_\triangle) = \check{T}^{\ell-1}(F) + \tilde{W}_\ell + \tilde{U}_\ell. \tag{Eq3'}$$

In order to investigate the local behavior of $\check{T}^{\ell-1}(F)$, we assume that all values from $F = T_{2^k}(X, Y)$ to $F_\triangle = T_{2^k}(X + \triangle_X,\, Y + \triangle_Y)$ are on the same surface in the graph of $T_{2^k}(X, Y)$, and all values from $\check{T}^j(F)$ to $\check{T}^j(F_\triangle)$, $j = 0, 1, \cdots, \ell - 1$, are on the same line in the graph of $\check{T}^j$. In this case, since $\tilde{K}_j(X, \triangle) = 0$ and hence $\tilde{U}_\ell = 0$, (Eq3') becomes

$$\check{T}^{\ell-1}(F_\triangle) - \check{T}^{\ell-1}(F) = \tilde{W}_\ell. \tag{Eq4'}$$

Therefore making use of $\tilde{W}_\ell$ we investigate below the relation between ℓ and randomness. From (Eq2') it holds that

$$\begin{aligned}\tilde{W}_\ell = {} & 2^{(\ell-1)(k+1)}\tilde{W}_1 \left\{\prod_{j=1}^{\ell-1} \check{T}^{j-1}(F)\right\}\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\} \\ = {} & 2^{(\ell-1)(k+1)+k}(X\triangle_Y + Y\triangle_X + \triangle_X\triangle_Y) \\ & \times \left\{\prod_{j=1}^{\ell-1} \check{T}^{j-1}(F)\right\}\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\}.\end{aligned} \tag{Eq5'}$$

Here we assume $\triangle_X$ and $\triangle_Y$ are positive. (This assumption is frequently satisfied by virtue of the way of defining $(X_i, Y_i)_i$ in §2.1, and can include the case of §6.1. It is not necessary that all pairs $(\triangle_X,\, \triangle_Y)$ of $(X + \triangle_X,\, Y + \triangle_Y)$ satisfy this assumption.) So we assume further that $\triangle_X = 2^{-n/t_X}$ and $\triangle_Y = 2^{-n/t_Y}$, t_X and $t_Y = 1$ or 2. These $\triangle_X$ and $\triangle_Y$ emulate roughly the positions where the result of taking XOR with P_i or Q_i appears.

[Case I] $(t_X = t_Y)$ Let's put $\tilde{t} \equiv t_X = t_Y$, and set $\tilde{\triangle} = 2^{-n/\tilde{t}}$, $\tilde{t} = 1,\, 2$. Then we have

$$(X\triangle_Y + Y\triangle_X + \triangle_X\triangle_Y) = \tilde{\triangle}(X + Y + \tilde{\triangle}) = 2\tilde{\triangle}\frac{X+Y}{2}(1 + \frac{\tilde{\triangle}}{X+Y}).$$

Therefore (Eq4')-(Eq5') for $k = (n/s) - 1$ becomes

$$\begin{aligned}& \check{T}^{\ell-1}(F_\triangle) - \check{T}^{\ell-1}(F) \\ & = 2^{\ell(n/s)-(n/\tilde{t})}\left[\frac{X+Y}{2}\left\{\prod_{j=1}^{\ell-1}\check{T}^{j-1}(F)\right\}\right] \times \left[(1 + \frac{\tilde{\triangle}}{X+Y})\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\}\right] \\ & = 2^{\ell((n/s)+\theta_{\ell,X,Y})-(n/\tilde{t})}\left[(1 + \frac{\tilde{\triangle}}{X+Y})\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\}\right].\end{aligned} \tag{Eq6'}$$

Now we apply the same argument given in §6.1, and have the expression $\check{\ell} = (s/\tilde{t})$. This result can explain the properties (O1)-(O4) when XORpos is CC($\tilde{t}$ =2) or TT($\tilde{t}$ =1) in Table 4 of §5.2 .

[Case II] $(t_X > t_Y)$ Let $t_X = 2$ and $t_Y = 1$, so $\triangle_X = 2^{-n/2}$ and $\triangle_Y = 2^{-n}$. Then

$$(X\triangle_Y + Y\triangle_X + \triangle_X\triangle_Y) = Y\triangle_X(\frac{X}{Y}\cdot\frac{\triangle_Y}{\triangle_X} + 1 + \triangle_Y/Y) = 2^{-n/2}Y(\frac{X}{Y}2^{-n/2} + 1 + 2^{-n}/Y).$$

Hence (Eq4')-(Eq5') for $k = (n/s) - 1$ becomes

$$\begin{aligned}& \check{T}^{\ell-1}(F_\triangle) - \check{T}^{\ell-1}(F) \\ & = 2^{\ell(n/s)-1-(n/2)}\left[Y\left\{\prod_{j=1}^{\ell-1}\check{T}^{j-1}(F)\right\}\right] \times \left[(\frac{X}{Y}2^{-n/2} + 1 + 2^{-n}/Y)\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\}\right] \\ & = 2^{\ell((n/s)+\theta_{\ell,X,Y})-1-(n/2)}\left[(1 + \frac{X}{Y}2^{-n/2} + 2^{-n}/Y)\left\{\prod_{j=1}^{\ell-1}\{1 + \tilde{W}_j/2\check{T}^{j-1}(F)\}\right\}\right].\end{aligned} \tag{Eq6"}$$

Since

$$\ell\{(n/s)+\theta_{\ell,X,Y}\}-1-(n/2)\geq 0 \;\Leftrightarrow\; \ell\geq \frac{s}{2}\left(1-\frac{(s/n)\theta_{\ell,X,Y}}{1+(s/n)\theta_{\ell,X,Y}}\right)+\frac{(s/n)}{1+(s/n)\theta_{\ell,X,Y}}, \qquad \text{(Eq7")}$$

repeating the same consideration as (Eq7), we have the expression $\check{\ell}=s/2(=s/t_X)$, which implies that randomness caused by 'C'($t_X=2$) dominates randomness due to 'T'($t_Y=1$) . This result can explain the property (O_5) when XORpos is CT in Table 4 of §5.2. The case $t_X<t_Y$ is treated similarly.

Finally, it should be noted here that our MmS corresponds to the case of $k=\frac{n}{s}-1$, $s=2$, and $t_X=t_Y=2$, and hence is characterized by [Case I], which implies $\check{\ell}=(s/\tilde{t})=1$. This expression says that randomness of MmS would appear apparently around $\tau_{\text{mul}}=1$ regardless of the value of n. (In fact the result 'o' of (S2) m_i(its τ_{mul} is 1) for $n=64$ and 128 in Table 1 can be explained well by $\check{\ell}=1$.) As the τ_{mul} of our MmS is 2((S2) and (S4)), the assertion $\check{\ell}=1$ above, which is independent of n, implies that the algorithm of MmS works well regardless of the value of n.

Remark 4. If one needs a *recursive* RNG(random number generator) using $T_{2^k}(X,Y)$, one can use, for example, f.b.l.com of $Z_i=T_{2^{(n/2)-1}}(Z_{i-1},Z_{i-1}\oplus(i-1))$, $i=1,2.\cdots$, and $Z_0=B$. (As a RN z_i, an n-bit number which corresponds to m in (S2) of §2.1 is used.) The operation $\oplus(i-1)$ (see also §2.1) is imposed in order to avoid entering a loop of generated RNs. This RNG is a kind of von Neumann's type, and passes the statistical test Crush stated in §2.2. (We will call this RNG the r.MmS(recursive MmS).)

Remark 5. Sample codes of implementing the algorithm of MmS in C are found in [4] as MMS***.c .

References

[1] P. L'Ecuyer and R. Simard, TestU01: A C library for empirical testing of random number generators, ACM Trans. Math. Software 33 (2007), no. 4, Paper No. 22. 10.1145/s1268776.1268777S

[2] H. Yaguchi, Construction of a nonrecursive 64-bit pseudorandom number generator based on beta transformations on [1,2), Departmental Bulletin Paper, Mie University, 2014, http://hdl.handle.net/10076/13947.

[3] H. Yaguchi, Generation of nonrecursive n-bit pseudorandom numbers based on β-transformation on $[1,2)$ (n =64, 128, 192, ..., 8192), Monte Carlo Methods Appl. 31 (2025), no. 3, 257-263. 10.1515/MCMA-2025-2015

[4] H. Yaguchi, https://sunday-diy.github.io/mms-rng .